The workflow for git-based z/OS DevOps

Recommended by the DAT Team

Ian J Mitchell Dennis Behm Mathieu Dalbin

Table of contents

0.1 Agenda . . . 2

1 Aims and Assumptions **2**
1.1 No baggage . . . 2
1.2 Scaling . . . 3
1.3 Planning . . . 3
1.4 Convention over configuration . . . 3

2 The Branching Strategy **4**
2.1 Starting Simple . . . 4
2.2 Starting Simple . . . 4
2.3 Preparing to Merge into `main` . . . 5
2.4 Merging into `main` . . . 5
2.5 Before you ask... no, there is no *Production* branch . . . 6
2.6 Testing a release candidate . . . 6
2.7 Deploying to production . . . 7
2.8 Release maintenance branches . . . 8
2.9 Scaling up . . . 8
2.10 Integration branches . . . 9

3 Workflows supported by the strategy **9**
3.1 The types of workflows . . . 10
3.2 Features for the next planned release . . . 10
3.3 Features for the next planned release . . . 10
3.4 Hot-fix for production . . . 11
3.5 Building the fix package . . . 12
3.6 Using the Epic branch workflow . . . 12
3.7 Epic branch workflow . . . 13
3.8 Guidelines and policies in the epic branch workflow . . . 13

4 Pipeline implementations **13**
4.1 Pipeline principles 13
4.2 Basic Build pipeline 14
4.3 Release pipeline 14

5 Collateral **14**

0.1 Agenda

- Aims and Assumptions
- Fundamentals of the branching strategy
- Supported workflows
- Implementing pipelines

```
some code
```

1 Aims and Assumptions

Some aims and assumptions that guide our recommendations...

1.1 No baggage

Well... *"travel light"* perhaps!

- Are we *prescriptive* or just *opinionated*?
  - Aim to be *git-native*!
- We start with a recommendation
  - Confidently
- We question everything
  - **YAGNI** - "you aren't gonna need it"
- We strive for simplicity
  - For each user's experience

1.2 Scaling

- The workflow and branching scheme should both scale up and scale down.
  - Small teams with simple and infrequent changes will be able to easily understand, adopt, and have a good experience.
  - Large, busy teams with many concurrent activities will be able to plan, track, and execute with maximum agility using the same fundamental principles.

1.3 Planning

- Planning and design activities as well as code development aim to align to a regular release cadence.
- The cadence of the next planned release is defined by the application team.
- There is no magic answer to managing large numbers of "in-flight" changes, so planning assumptions should aim as much as possible to complete changes quickly, ideally within one release cycle.
  - DevOps/Agile practices to break down larger changes into sets of **smaller, incremental** deliverables that can each be completed within an iteration.
  - This reduces the number of "in-flight" changes, and allows the team to deliver **value** more quickly while still building towards a larger development goal.
- We know it is sometimes unavoidable for work to take longer than one release cycle and we accommodate that as a variant of the base workflow.

1.4 Convention over configuration

The principles are more important that the tools and names.

Naming conventions - making purpose obvious:

- `main`: single source of truth
- `release/rel-2.0.1`: explicit versioning numbering of releases
- `feature/<jira-id|servicenow-id>/new-mortage-calculation`: traceable to other processes
- `epic/ai-fraud-detection`: maybe put Jira or ServiceNow ids in here too
- `hotfix/rel-2.0.1/fix-mortgage-calc`

Names of branches and tags flow through to builds and deployments.

2 The Branching Strategy

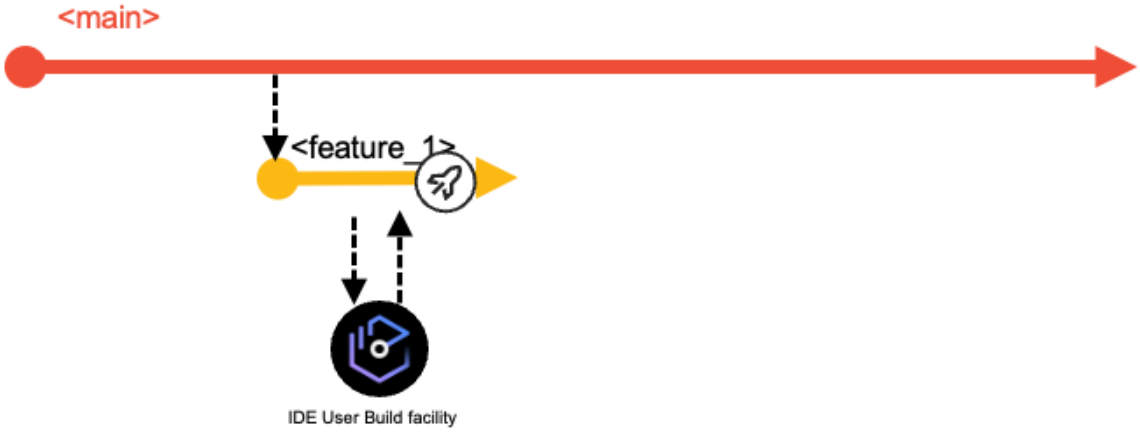
2.1 Starting Simple

Every change starts in a branch.

- Developers work in the branch to make changes, perform user builds and unit tests.
- A branch holds multiple commits (changes to multiple files).

2.2 Starting Simple

Every change starts in a branch.



These changes on these branches are

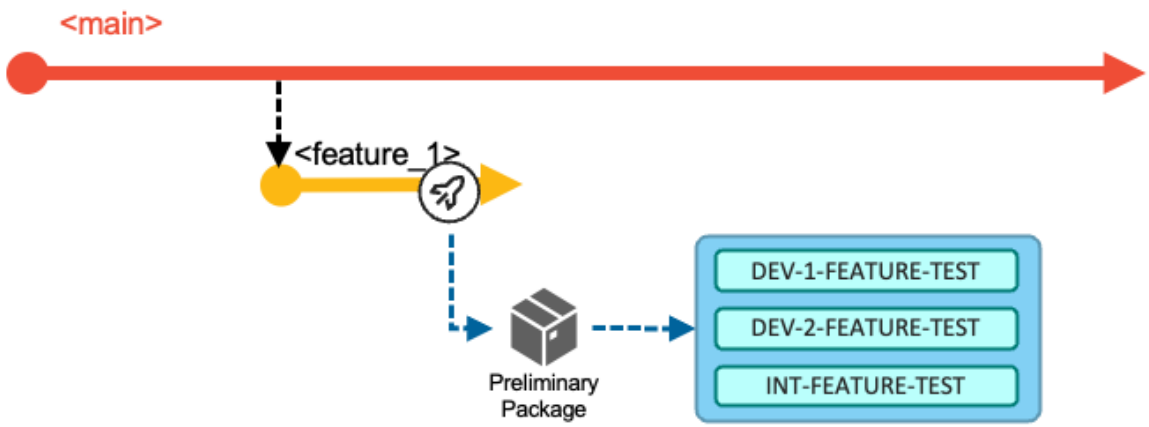
- built,
- tested,
- reviewed and

- approved before merging to `main`.

2.3 Preparing to Merge into `main`

Feature Team/Developers will:

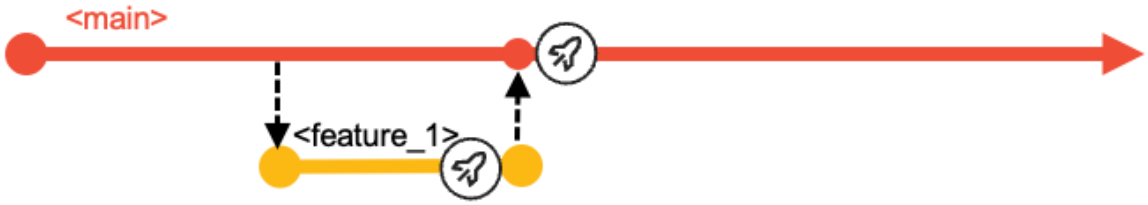
- Build
  - Builds may be done to any commit on any branch
  - Feature branch **must** build cleanly for a Pull Request
- Test
  - To prove quality of the changes in their feature branch



2.4 Merging into `main`

Create a Pull Request (PR) to signal to Team Leaders/Release Controllers to:

- Review
  - Code and Test results
- Approve
  - Safeguard the quality of `main`



`main` and other long-lived branches are *protected* (not everyone can `push` or `merge` to them).

2.5 Before you ask... no, there is no *Production* branch

CI/CD decouples the building and deploying to test environments and production.

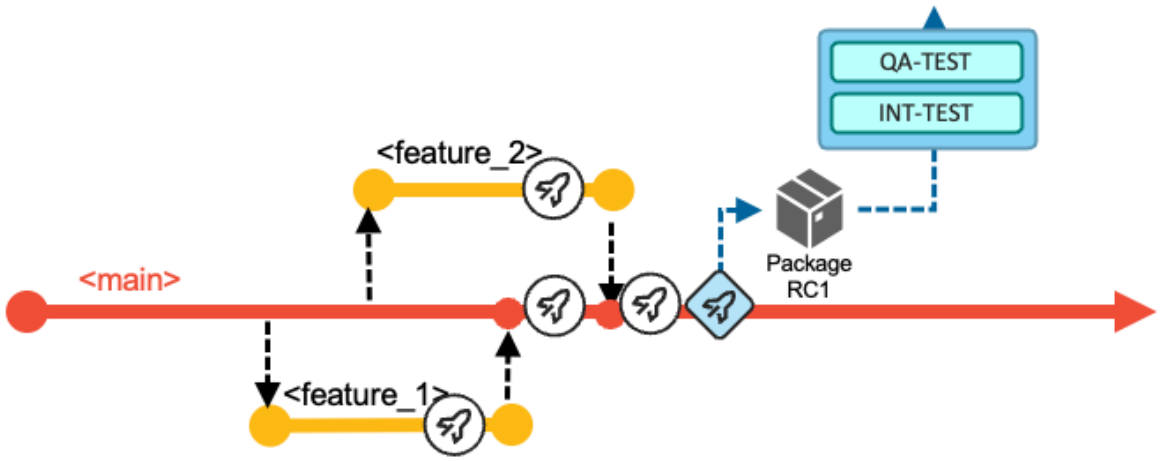
We have no branches named `prod` (or `test` or `QA`)

- Those are *environments* to which builds can be deployed
- Such extra branches:
  - are unnecessary
  - cause ambiguity
  - impose merging and building overheads
- Deployment manager is maintaining what is deployed where and provides traceability from developer to production deployment

2.6 Testing a release candidate

Any point in the history of `main` can be declared a *release candidate*.

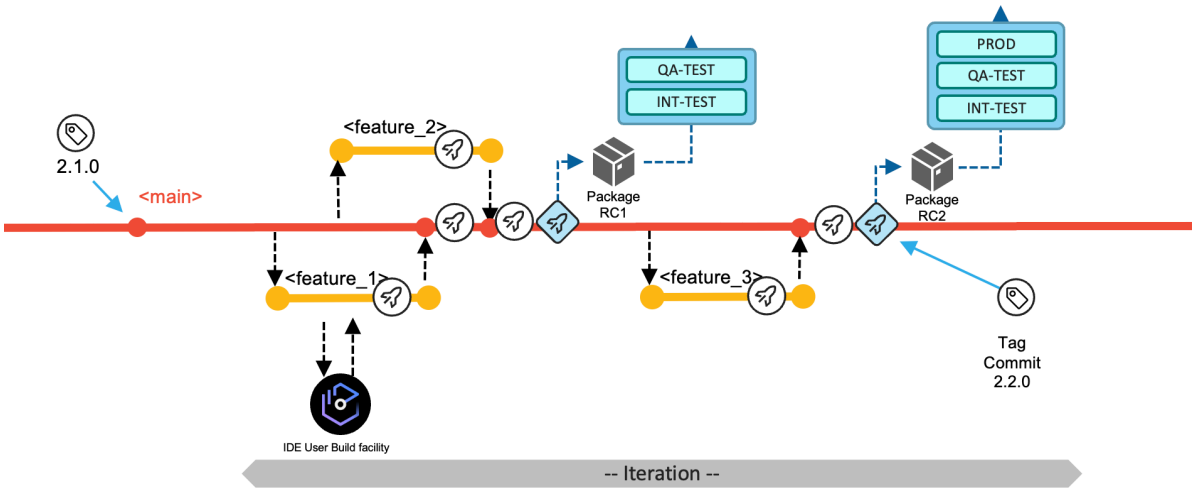
- Build a *release candidate* package
- Deploy it
- Test it



Tag the commit (point in `main`'s history) with a release name.

2.7 Deploying to production

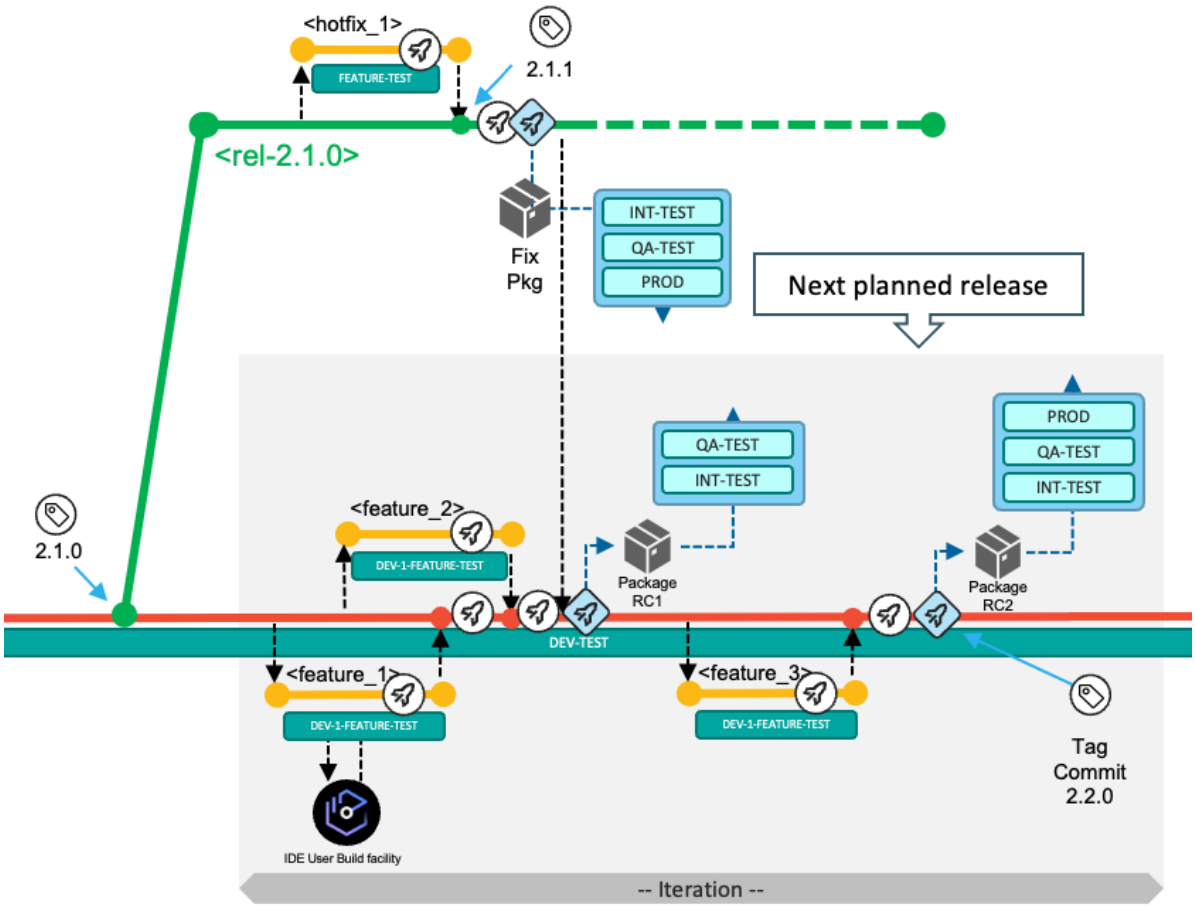
When all the committed work items for the next planned release are ready to be shipped, and all quality gates have passed successfully and the required approvals have been issued by the appropriate reviewers, the release package and be deployed to production.



Tag the commit (point in `main`'s history) with a release name.

2.8 Release maintenance branches

A *release maintenance* branch will be used if *hot-fixes* must be developed and delivered.



2.9 Scaling up

Concurrent *feature* branches scale very well, but assume short cycle times.

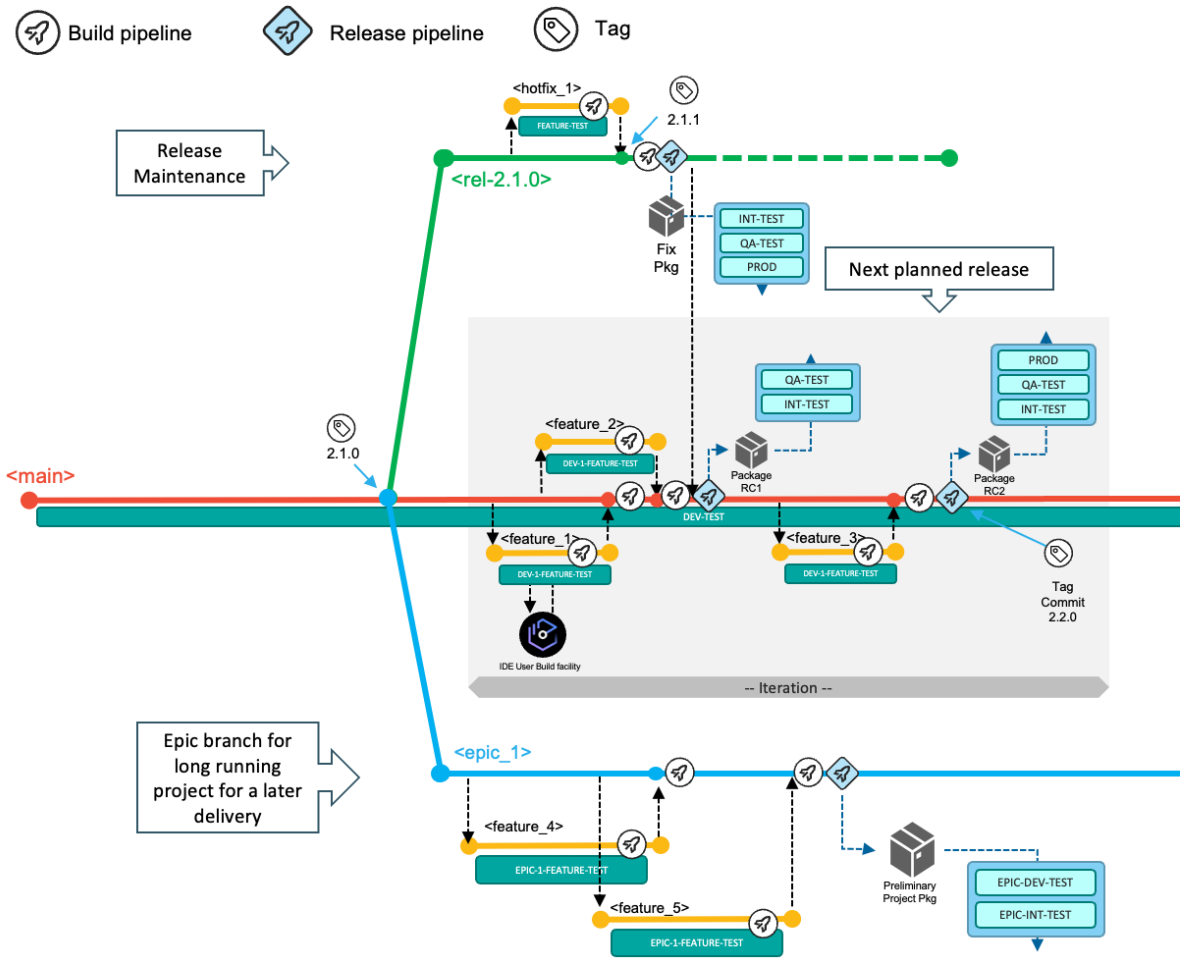
- Ideally live within a release delivery cycle
- But no big deal if they don't

Epic branches can collect multiple features

- Before going to `main`
- When the delivery is planned beyond the next release

(*Epic* branches are a form of *integration* branch.)

2.10 Integration branches



3 Workflows supported by the strategy

How do teams work with and benefit from the branch strategy?

3.1 The types of workflows

1) Work and focus on the **next planned release** via the `main` branch. After planning the work items for the next release, the development team is adding changes to the main branch.
2) **Resolution of a production problem** in the currently-released version of the application by leveraging a release maintenance branch that is used for maintenance purposes,
3) **concurrent development activities for significant development initiatives**, which include multiple planned work items for a later delivery (including starting development of a future release) by creating an epic branch from a commit point in the history of main.

The following narratives complement the structure.

3.2 Features for the next planned release

Developer activities:

1. Take item from backlog
2. Create feature branch
3. Code locally
4. Build with DBB User Build
5. Commit into feature branch, build and test feature functionality
6. Create Pull Request for review and approval

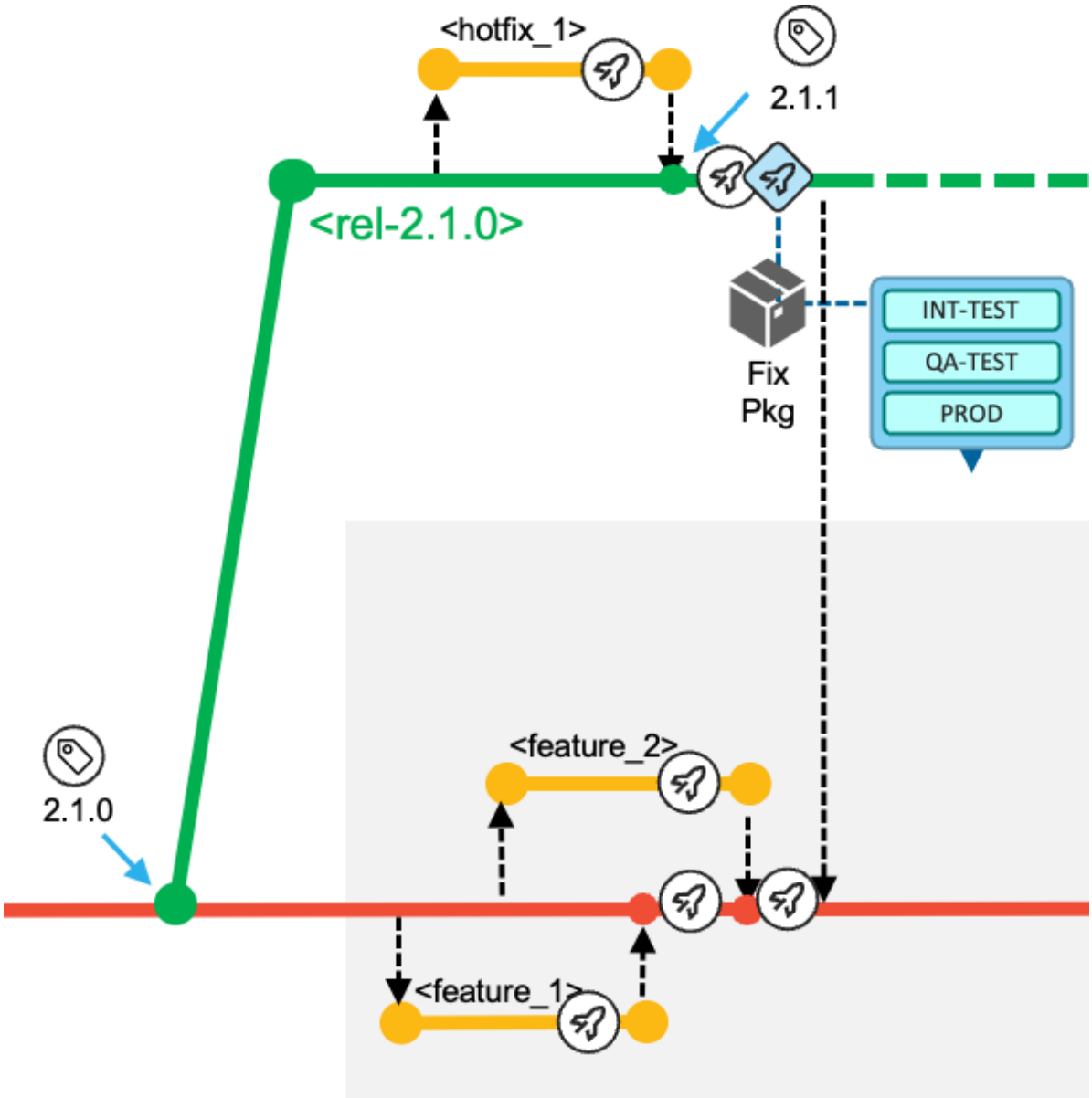
3.3 Features for the next planned release

Release control activities:

7. Merge approved branches to `main` and build
8. When planned features have merged, build and tag Release Candidate package
9. Deploy Release Candidate to test environments
10. (*Release candidate can be superceded*)
11. Finalise Release package in Artifactory (or equivalent)
12. Tag commit from step 8 as the production release and deploy

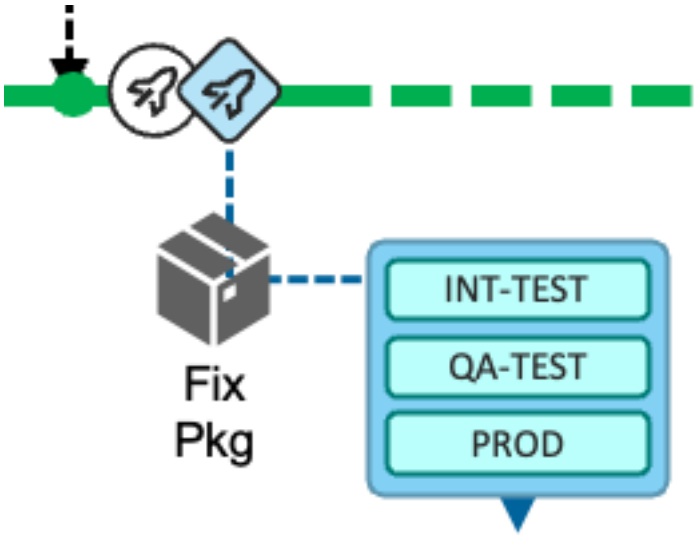
3.4 Hot-fix for production

- The process of urgent fixes for modules in the production environment follows the fix-forward approach, rather than rolling back the affected modules and reverting to the previous deployed state.
- The development team starts this process by creating a **release maintenance** branch of the git tag that represents the most recent release, that was deployed to production.



3.5 Building the fix package

- Changes contributed to the release maintenance branch are build and packaged.
- The fix package is tested in the appropriate test environment(s)
- Finally deployed to production



- The commit is tagged as the new version `2.1.1`.

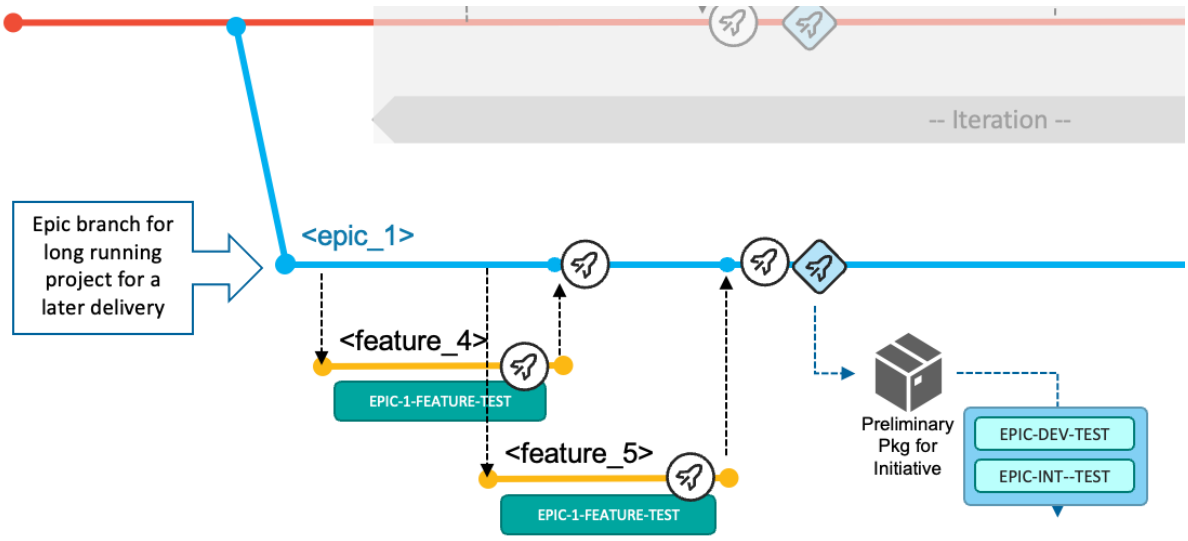
3.6 Using the Epic branch workflow

Dealing with significant development initiatives such as:

- Updates in regulatory frameworks
- Start the development phase of *subsequent* releases

→ A process to work independently from the default workflow.

3.7 Epic branch workflow



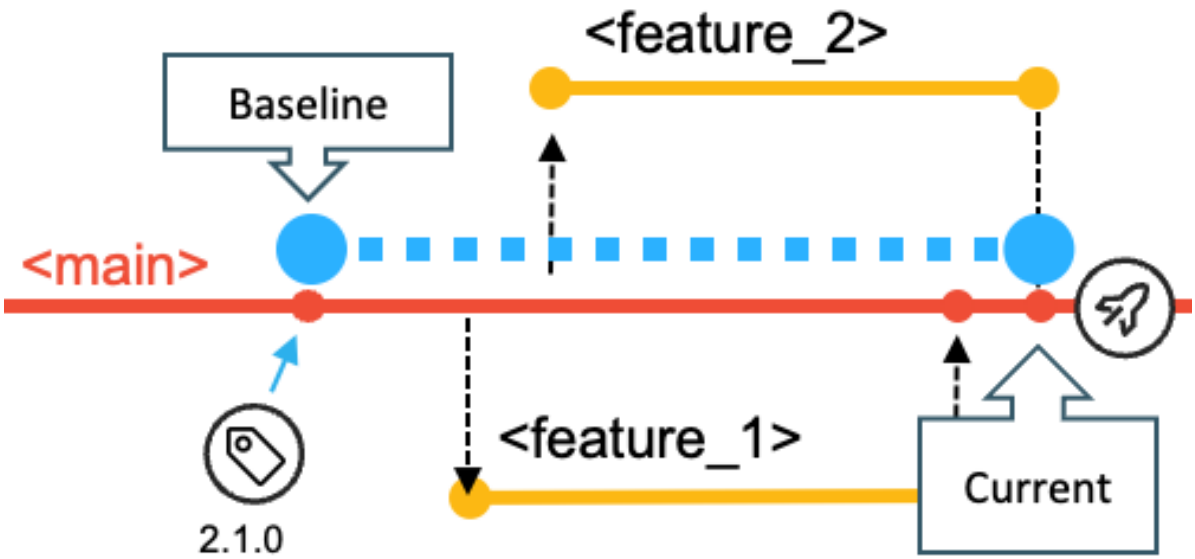
3.8 Guidelines and policies in the epic branch workflow

- Frequently synchronize with `main` into the epic branch to avoid making the merge of the `epic` branch into `main` difficult.
- Preliminary packages must not be deployed to a production runtime.

4 Pipeline implementations

4.1 Pipeline principles

Git tags are used to calculate the changes for the next planned release



Packages are always cumulative.

4.2 Basic Build pipeline

- (Typically) runs on each commit added to the history of a branch
- Produces a preliminary package that is stored in a binary repository
- May include an automated step to deploy it into the DEV-TEST environment

4.3 Release pipeline

- Produces binaries optimized for performance
- (Typically) is manually requested by the development team when a level of stability is reached
- Produces a release candidate package that can be deployed to higher test environments.

5 Collateral